4.3 Vector 方言到 NVGPU 方言的转换

Vector 方言到 NVGPU 方言的转换

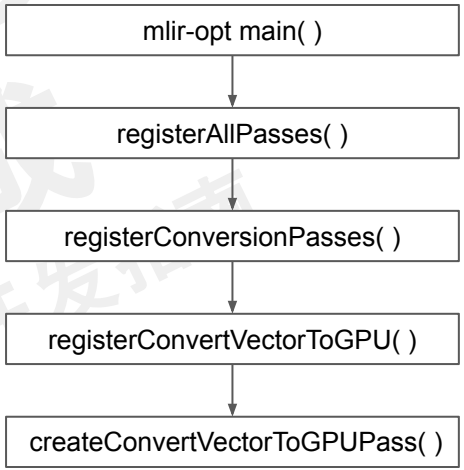
- vector.transfer_read 操作到 nvgpu.ldmatrix 操作和非 nvgpu.ldmatrix 操作的转换
- vector.contract 操作到 nvgpu.mma.sync 操作的转换
- vector.transfer_write 操作的转换

4.3.1 ConvertVectorToGPU pass 的定义与实现

ConvertVectorToGPU pass 的定义与实现

Passes.td

```
def ConvertVectorToGPU : Pass<"convert-vector-to-gpu"> {
  let summary = "Lower the operations from the vector dialect into the
GPU dialect";
  let constructor = "mlir::createConvertVectorToGPUPass()";
  let dependentDialects = [
    "memref::MemRefDialect", "gpu::GPUDialect", "AffineDialect",
    "vector::VectorDialect", "nvgpu::NVGPUDialect"
  ];
  let options = [
    Option<"useNvGpu", "use-nvgpu", "bool", /*default=*/"false", "...">
  ];
}
```



ConvertVectorToGPU pass 的定义与实现

VectorToGPU.cpp

```
struct ConvertVectorToGPUPass
    : public impl::ConvertVectorToGPUBase<ConvertVectorToGPUPass> {
  explicit ConvertVectorToGPUPass(bool useNvGpu_) {
    useNvGpu.setValue(useNvGpu_);
  }

  void runOnOperation() override {
    RewritePatternSet patterns(&getContext());
    populatePrepareVectorToMMAPatterns(patterns, useNvGpu.getValue());
    if (failed(applyPatternsAndFoldGreedily(getOperation(), std::move(patterns))))
      return signalPassFailure();

    if (useNvGpu.getValue()) {
      if (failed(convertVectorToNVVMCompatibleMMASync(getOperation())))
        return signalPassFailure();
    }
    (void)convertVectorToMMAOps(getOperation());
  }
};
```

4.3.2 Vector 操作到 NVGPU 操作的转换框架

Vector 操作到 NVGPU 操作的转换框架

```
LogicalResult mlir::convertVectorToNVVMCompatibleMMASync(RewriterBase &rewriter,
                                                         Operation *rootOp) {
  SetVector<Operation *> ops = getOpToConvert(rootOp, /*useNvGpu=*/true);
  llvm::DenseMap<Value, Value> valueMapping;
  for (Operation *op : ops) {
    if (llvm::TypeSwitch<Operation *, LogicalResult>(op)
          .Case([&](vector::TransferReadOp transferReadOp) {
            return convertTransferReadToLoads(rewriter, transferReadOp, valueMapping);
          })
          .Case([&](vector::TransferWriteOp transferWriteOp) {
            return convertTransferWriteToStores(rewriter, transferWriteOp, valueMapping);
          })
          …
          .Case([&](vector::ContractionOp contractionOp) {
            return convertContractOpToMmaSync(rewriter, contractionOp, valueMapping);
          })
      …
    }
  }
  return success();
}
```

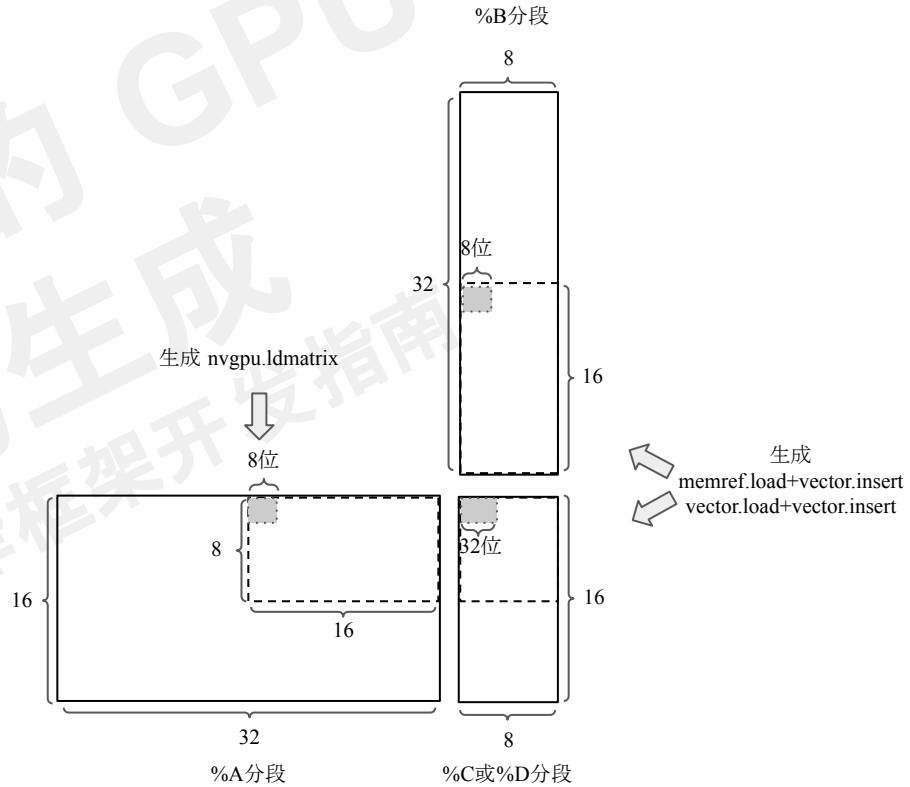
```
%A = vector.transfer_read %arg0…
%B = vector.transfer_read %arg1…
%C = vector.transfer_read %arg2…
%D = vector.contract {...} %A, %B, %C : …
vector.transfer_write %D, %arg2…
```

测试用例

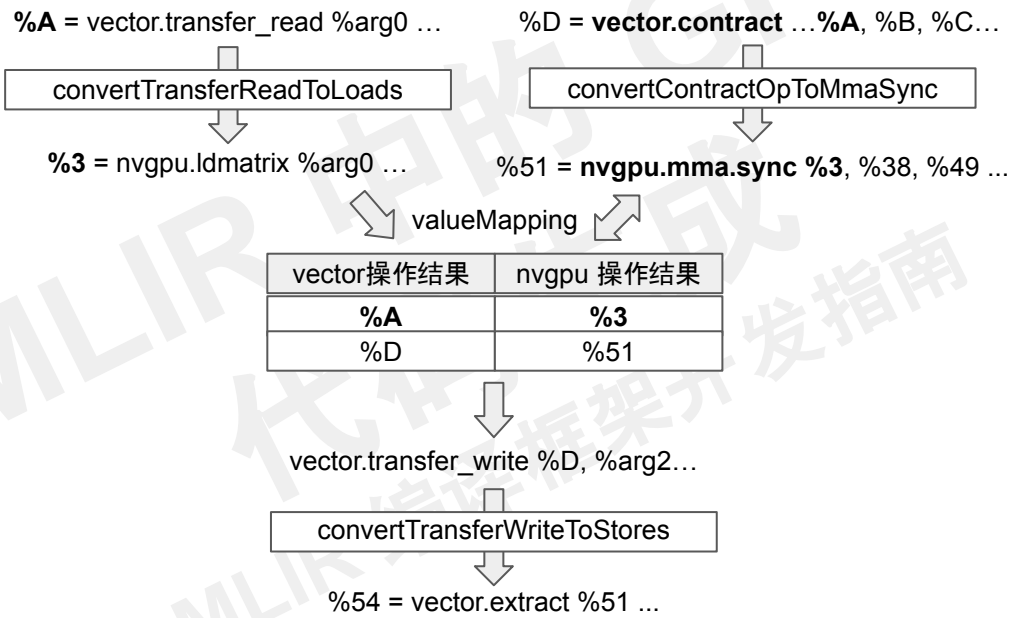
测试用例 vector-to-nvgpu.mlir

```
#map0 = affine_map<(d0, d1) -> (d1, d0)>
#map1 = affine_map<(d0, d1, d2) -> (d0, d2)>
#map2 = affine_map<(d0, d1, d2) -> (d1, d2)>
#map3 = affine_map<(d0, d1, d2) -> (d0, d1)>
```

```
  %A = vector.transfer_read %arg0[%c0, %c0], %cst {in_bounds = [true, true]} :
memref<128x128xi8, #gpu.address_space<workgroup>>, vector<16x32xi8>
  %B = vector.transfer_read %arg1[%c39, %c40], %cst {in_bounds = [true, true],
permutation_map = #map0} : memref<128x128xi8, #gpu.address_space<workgroup>>,
vector<8x32xi8>
  %C = vector.transfer_read %arg2[%c49, %c40], %cst0 {in_bounds = [true, true]} :
memref<128x128xi32>, vector<16x8xi32>
  %D = vector.contract {indexing_maps = [#map1, #map2, #map3], iterator_types = ["parallel",
"parallel", "reduction"], kind = #vector.kind<add>} %A, %B, %C : vector<16x32xi8>,
vector<8x32xi8> into vector<16x8xi32>
  vector.transfer_write %D, %arg2[%c49, %c40] {in_bounds = [true, true]} : vector<16x8xi32>,
memref<128x128xi32>
```

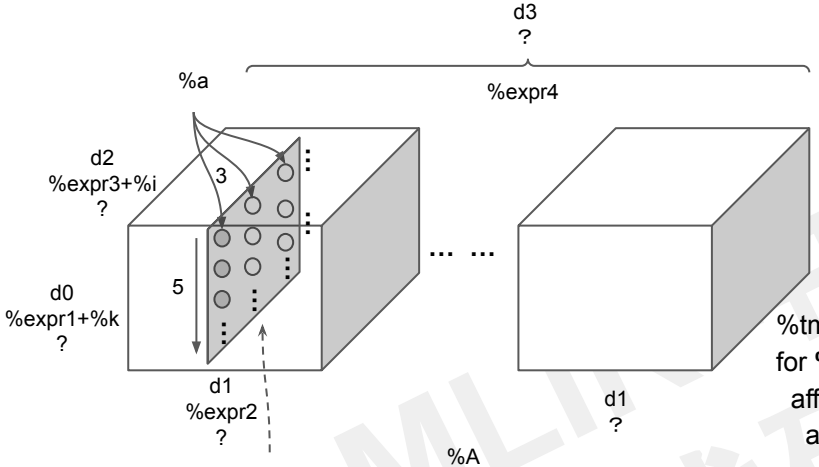


操作转换结果缓存



4.3.3 vector.transfer_read 操作的转换过程

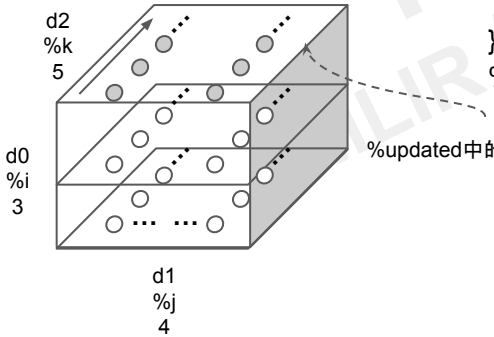
vector.transfer_read 操作功能



%A

vector.transfer_read

%updated

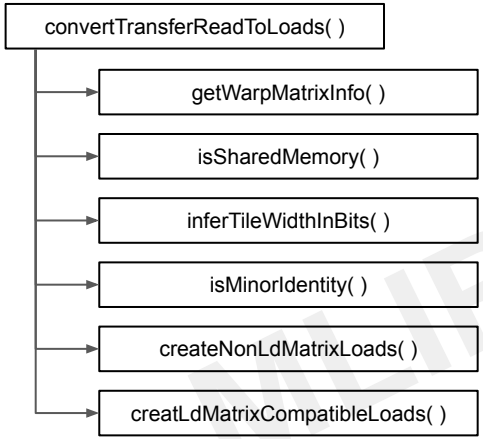



```
%vec = vector.transfer_read %A[%expr1, %expr2, %expr3, %expr4]
 { permutation_map : (d0,d1,d2,d3) -> (d2,0,d0) } :
 memref<?x?x?x?xf32>, vector<3x4x5xf32>
```

⇩ 等效代码

```
%tmp = memref.alloc() : memref<vector<3x4x5xf32>>
for %i = 0 to 3 {
  affine.for %j = 0 to 4 {
    affine.for %k = 0 to 5 {
      %a = load %A[%expr1 + %k, %expr2, %expr3 + %i, %expr4] : memref<?x?x?x?xf32>
      %slice = memref.load %tmp : memref<vector<3x4x5xf32>> -> vector<3x4x5xf32>
      %updated = vector.insert %a, %slice[%i, %j, %k] : f32 into vector<3x4x5xf32>
      memref.store %updated, %tmp : memref<vector<3x4x5xf32>>
}}}
%vec = memref.load %tmp : memref<vector<3x4x5xf32>> -> vector<3x4x5xf32>
```

convertTransferReadToLoads( ) 函数流程图




```
struct WarpMatrixInfo {
  VectorType vectorType;
  MatMulOperandRole operandRole;
};
```

%A = **vector.transfer_read** %arg0[%c0, %c0], %cst {in_bounds = [true, true]} :
memref<128x128xi8, #gpu.address_space<workgroup>>, **vector<16x32xi8>**
…
%D = **vector.contract** {indexing_maps = [#map1, #map2, #map3], …} **%A**, %B, %C :
vector<16x32xi8>, vector<8x32xi8> into **vector<16x8xi32>**
vector.transfer_write %D, %arg2[%c49, %c40] {in_bounds = [true, true]} :
vector<16x8xi32>, memref<128x128xi32>

vectorType = vector<16x32xi8>, operandRole = MatMulOperandRole::A

vectorType = vector<16x8xi32>, operandRole = MatMulOperandRole::C

vector.transfer_read 操作转换条件

vector.transfer_read 操作转换为 nvgpu.ldmatrix 操作的三个前置条件：

- 源操作数在共享内存中（isSharedMemory）
- warp级矩阵操作数类型对应的 8 × 8 分片单行宽度为 128 位（inferTileWidthInBits）
- vector.transfer_read 操作的交错映射为次要恒等映射（isMinorIdentity）

```
%A = vector.transfer_read %arg0[%c0, %c0], %cst {in_bounds = [true, true]} :
        memref<128x128xi8, #gpu.address_space<workgroup>>, vector<16x32xi8>
```

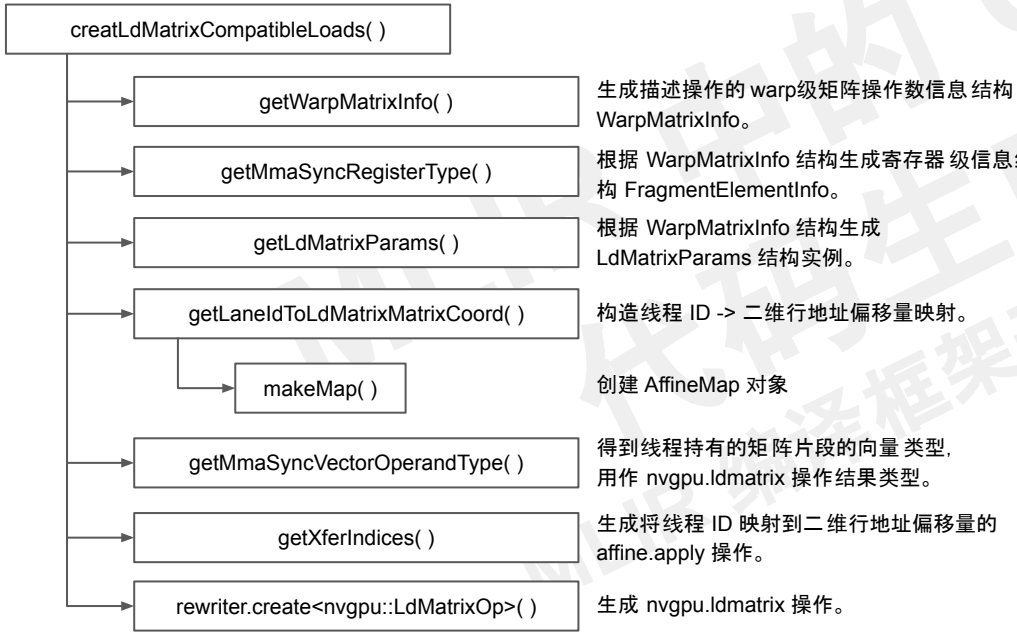
vector.transfer_read 操作转换条件

vector.transfer_read 操作转换为 nvgpu.ldmatrix 操作的三个前置条件：

- 源操作数在共享内存中（isSharedMemory）
- warp级矩阵操作数类型对应的 8 × 8 分片单行宽度为 128 位（inferTileWidthInBits）
- vector.transfer_read 操作的交错映射为次要恒等映射（isMinorIdentity）

```
#map0 = affine_map<(d0, d1) -> (d1, d0)>
...
 %A = vector.transfer_read %arg0[%c0, %c0], %cst {in_bounds = [true, true]} :
              memref<128x128xi8, #gpu.address_space<workgroup>>, vector<16x32xi8>
 %B = vector.transfer_read %arg1[%c39, %c40], %cst {in_bounds = [true, true], permutation_map = #map0} :
              memref<128x128xi8, #gpu.address_space<workgroup>>, vector<8x32xi8>
 %C = vector.transfer_read %arg2[%c49, %c40], %cst0 {in_bounds = [true, true]} : memref<128x128xi32>, vector<16x8xi32>
...
```

vector.transfer_read 操作到 nvgpu.ldmatrix 操作的转换



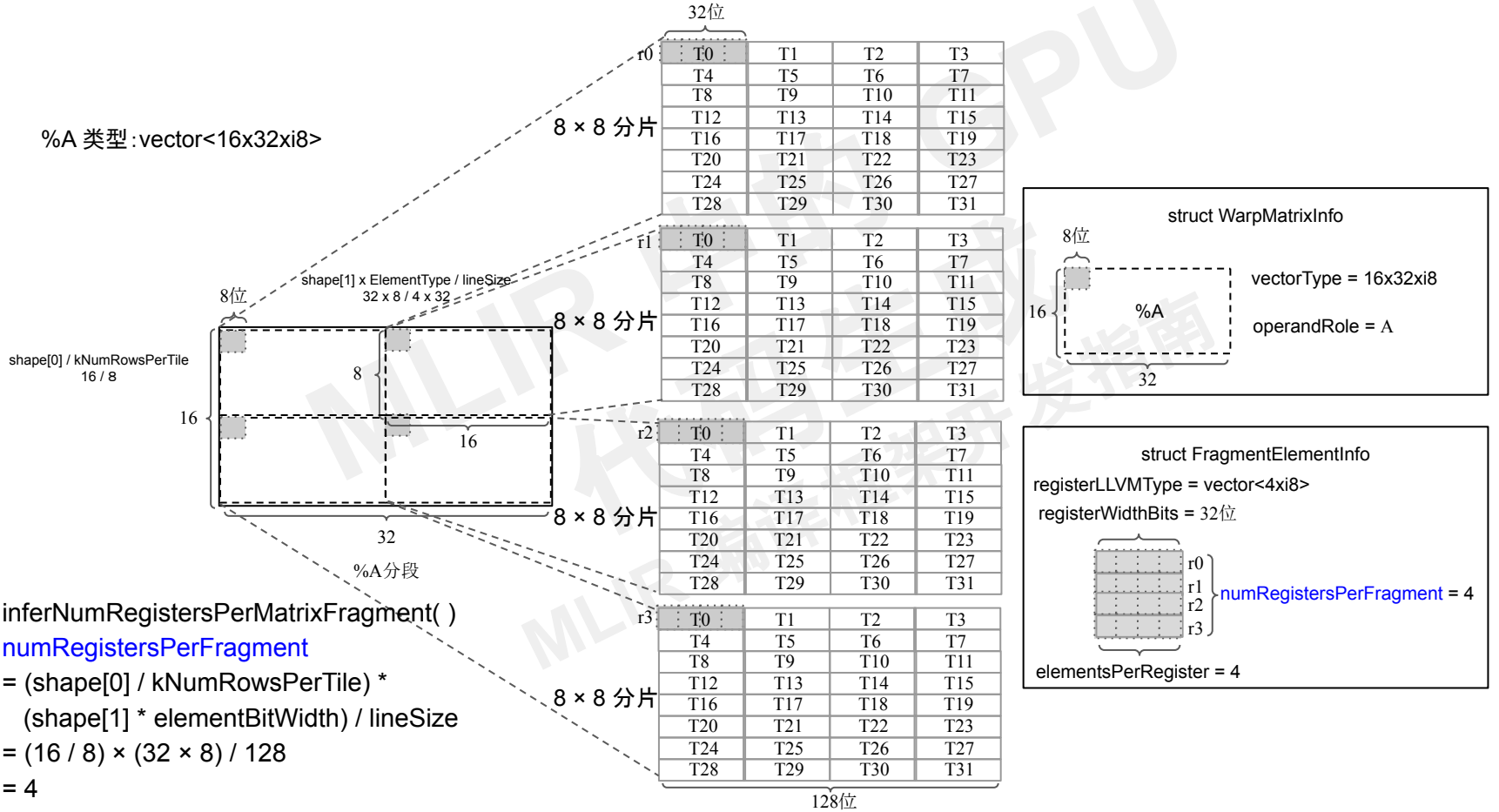

```
struct WarpMatrixInfo {
  VectorType vectorType;
  MatMulOperandRole operandRole;
};

struct FragmentElementInfo {
  Type registerLLVMType;
  int64_t elementsPerRegister;
  int64_t registerWidthBits;
  int64_t numRegistersPerFragment;
};

struct LdMatrixParams {
  VectorType fragmentType;
  bool isAccum;
  int64_t numTiles;
  vector::IteratorType contiguousDimType;
  NVVM::MMALayout targetLayout;
};
```

操作数布局与 8 × 8 分片布局的对应关系及相关数据结构

%A 类型:vector<16x32xi8>



inferNumRegistersPerMatrixFragment( )

numRegistersPerFragment

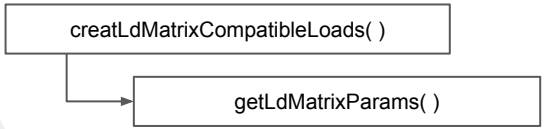
= (shape[0] / kNumRowsPerTile) *

(shape[1] * elementBitWidth) / lineSize

= (16 / 8) × (32 × 8) / 128

= 4

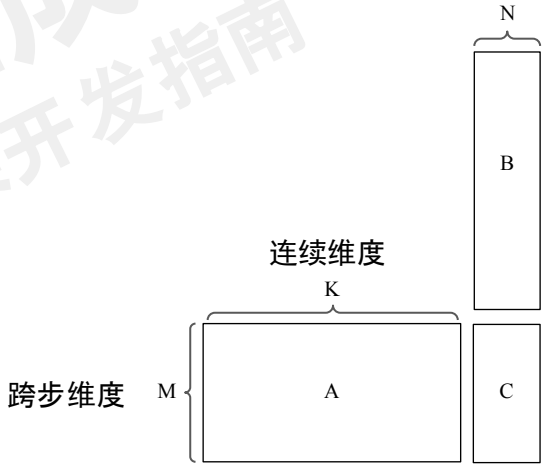
getLdMatrixParams 说明




```
FailureOr<nvgpu::LdMatrixParams>
nvgpu::getLdMatrixParams(const WarpMatrixInfo &type, bool transpose) {
  LdMatrixParams params;
  Type elType = type.vectorType.getElementType();
  params.fragmentType = type.vectorType;
  if (type.operandRole == MatMulOperandRole::A ||
      type.operandRole == MatMulOperandRole::C) {
    params.targetLayout = NVVM::MMALayout::row;
  } else {
    params.targetLayout = NVVM::MMALayout::col;
  }
  ArrayRef<int64_t> shape = type.vectorType.getShape();
  params.contiguousDimType = transpose ? vector::IteratorType::parallel
                       : vector::IteratorType::reduction;

  if (params.contiguousDimType == vector::IteratorType::reduction) {
    params.numTiles = (shape[0] / kNumRowsPerTile) *
                      ((shape[1] * elType.getIntOrFloatBitWidth()) / 128);
  } else {
    params.numTiles = (shape[1] / kNumRowsPerTile) *
                      ((shape[0] * elType.getIntOrFloatBitWidth()) / 128);
  }
  if (params.numTiles == 0)
    return failure();
  return params;
}
```

```
struct LdMatrixParams {
  VectorType fragmentType;
  bool isAccum;
  int64_t numTiles;
  vector::IteratorType contiguousDimType;
  NVVM::MMALayout targetLayout;
};
```

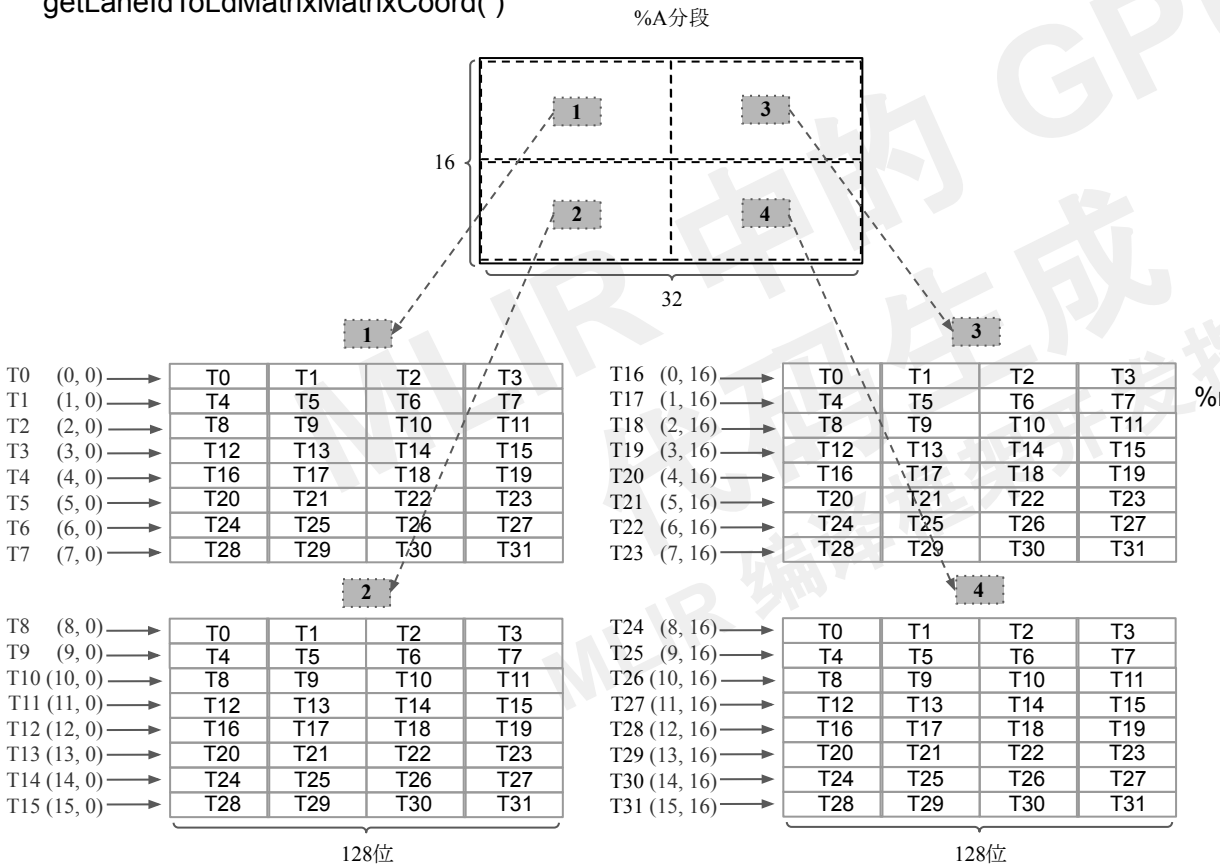


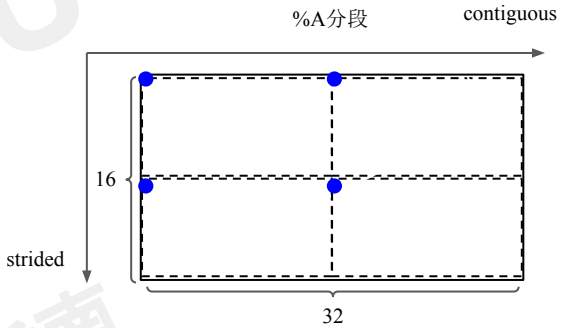
线程 ID 到 (stride, contiguous) 坐标的映射

creatLdMatrixCompatibleLoads( )

getLaneIdToLdMatrixMatrixCoord( )

getLaneIdToLdMatrixMatrixCoord( )



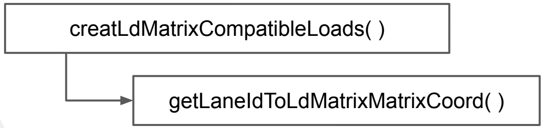



```
%res = nvgpu.ldmatrix %srcMemref[%indices]
                {numTiles = ..., transpose = ...} :
                type(%srcMemref) -> type(%res)
```

strided = laneid % 16

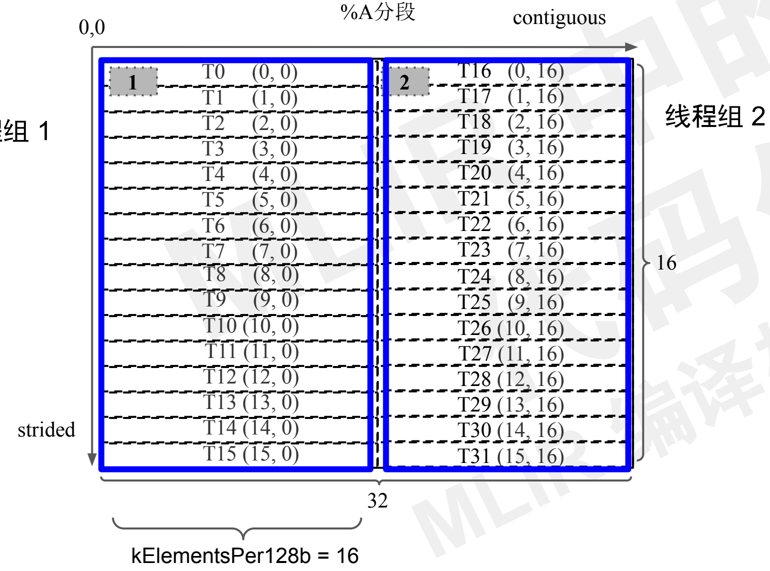
contiguous = (laneid / 16) * 16

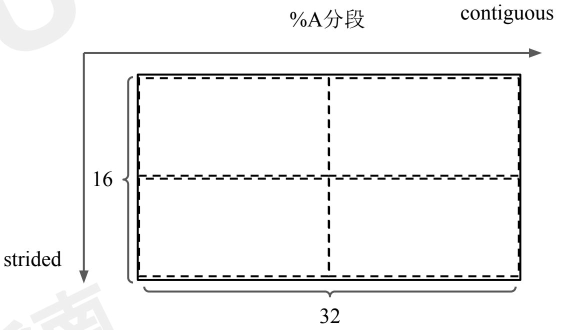
(stride, contiguous) 坐标点在 %A 中的位置



getLaneIdToLdMatrixMatrixCoord( )

%A 类型:vector<16x32xi8>





跨步维度索引　连续维度索引

(laneid) -> (strided , contiguous)

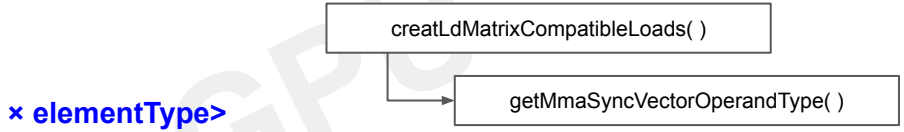
(d0) -> (d0 mod 16, (d0 floordiv 16) * 16)

```
AffineExpr strided = d0 % (operandShape[idx]);
AffineExpr contiguous = d0.floorDiv(operandShape[idx]) *
                        (kElementsPer128b);
```

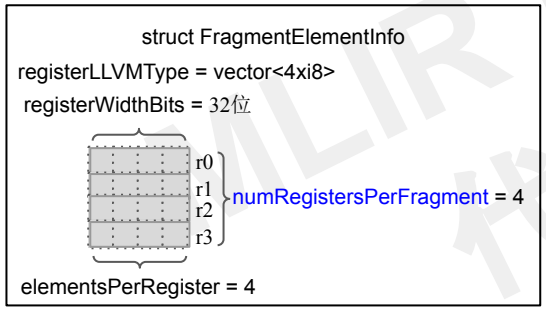
operandShape = [16 ,32]

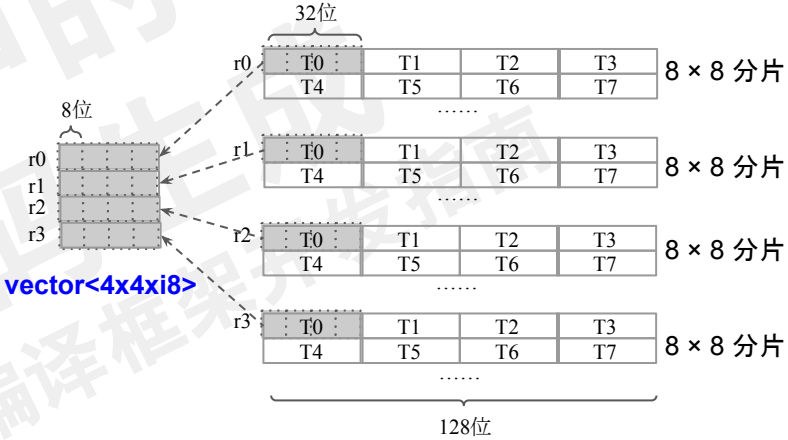
线程持有的数据片段

getMmaSyncVectorOperandType( )
vectorType
= **vector<numRegistersPerFragment × elementsPerRegister × elementType>**
= **vector<4x4xi8>**



%A 类型:vector<16x32xi8>






```
%res = nvgpu.ldmatrix %srcMemref[%indices]
                {numTiles = ..., transpose = ...} :
                type(%srcMemref) -> type(%res)
```

应用线程 ID 到矩阵地址偏移量映射

creatLdMatrixCompatibleLoads( )

contiguous

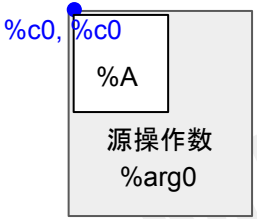
getXferIndices( )

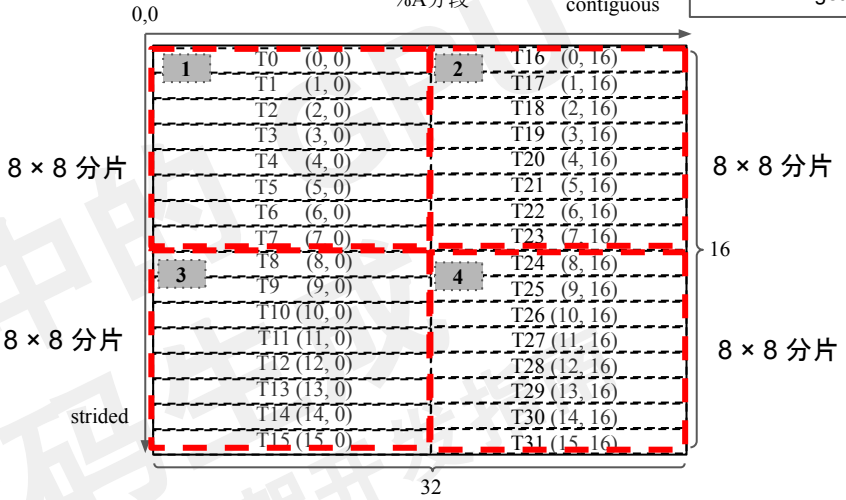
getXferIndices( ) 函数参数：

offsetMap = (d0) -> (d0 mod 16, (d0 floordiv 16) * 16)

dimValues = {laneId}

indices = [%c0, %c0]





%A = **vector.transfer_read** %arg0[%c0, %c0], … :
memref<128x128xi8, #gpu.address_space<**workgroup**>>,
vector<16x32xi8>

%res = nvgpu.ldmatrix %srcMemref[%indices]
{numTiles = ..., transpose = ...} :
type(%srcMemref) -> **type(%res)**

permutationMap = (d0, d1) -> (d0, d1)

#map0 = affine_map<()[s0] -> (s0 mod 16)>
#map1 = affine_map<()[s0] -> ((s0 floordiv 16) * 16)>
%0 = gpu.lane_id
%1 = affine.apply #map0()[%0]
%2 = affine.apply #map1()[%0]

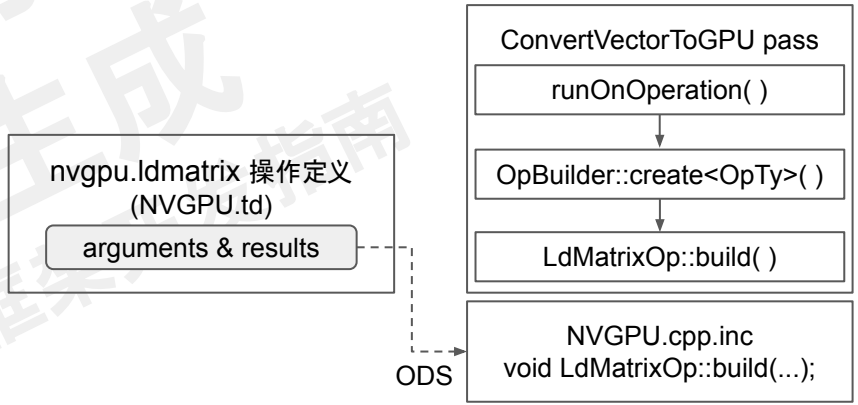
strided
%1 = affine.apply affine_map<()[s0] -> (s0 mod 16)>()[%0]

contiguous
%2 = affine.apply affine_map<()[s0] -> ((s0 floordiv 16) * 16)>()[%0]

生成 nvgpu.ldmatrix 操作

NVGPU.td

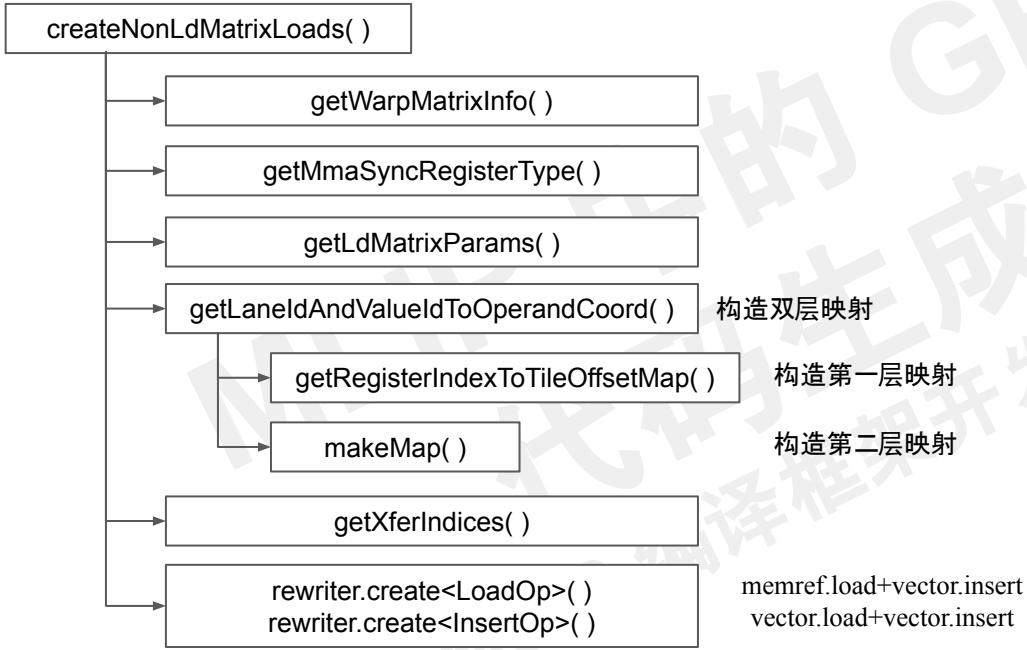
```
def NVGPU_LdMatrixOp : NVGPU_Op<"ldmatrix", …]> {
…
  let arguments = (ins Arg<AnyMemRef, "", [MemReadAt<0, FullEffect>]>:$srcMemref,
                       Variadic<Index>:$indices, BoolAttr:$transpose, I32Attr:$numTiles);
  let results = (outs AnyVector:$res);
…
}
```




```
#map0 = affine_map<()[s0] -> (s0 mod 16)>
#map1 = affine_map<()[s0] -> ((s0 floordiv 16) * 16 + 1)>
…
%0 = gpu.lane_id
%1 = affine.apply #map0()[%0]
%2 = affine.apply #map1()[%0]
%3 = nvgpu.ldmatrix %arg0[%1, %2] {numTiles = 4 : i32, transpose = false} :
memref<128x128xi8, #gpu.address_space<workgroup>> -> vector<4x4xi8>
```

```
%A = vector.transfer_read %arg0[%c0, %c0], … :  memref<128x128xi8, #gpu.address_space<workgroup>>, vector<16x32xi8>
```

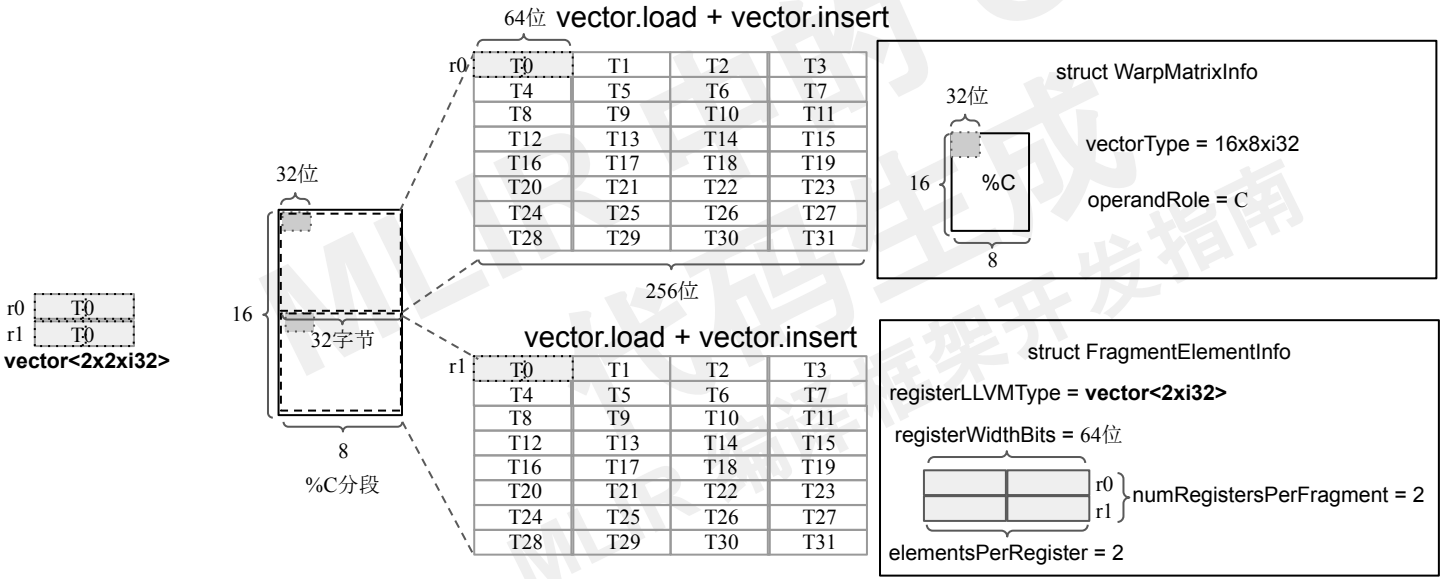
vector.transfer_read 操作到非 nvgpu.ldmatrix 操作的转换



举例：%C = vector.transfer_read %arg2[%c49, %c40], %cst0 … : memref<128x128xi32>, vector<16x8xi32>

加载 %C 的warp数据布局及相关数据结构

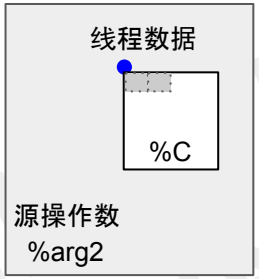
%C = **vector.transfer_read** %arg2[%c49, %c40], %cst0 {in_bounds = [true, true]} :
memref<128x128xi32>, **vector<16x8xi32>**

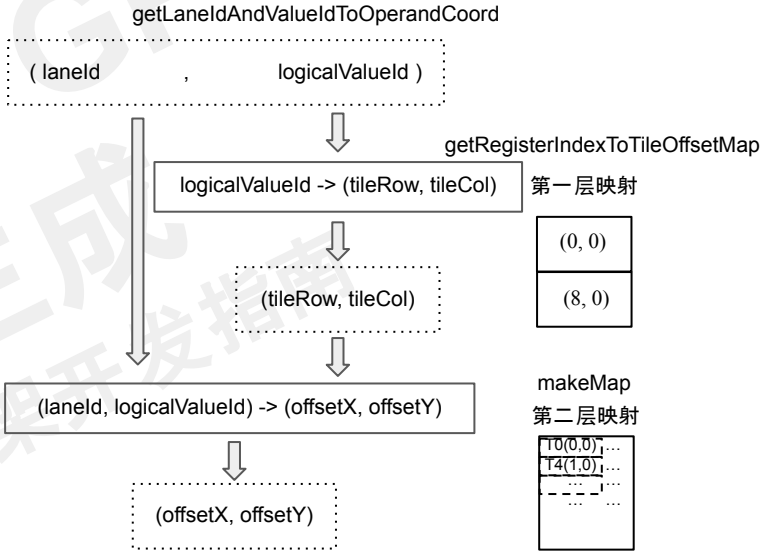


vector<2x2xi32>

vector<numRegistersPerFragment × elementsPerRegister × elementType>

双层映射结构





(laneId, logicvalueid) -> (tileRow, tileCol) -> (offsetX, offsetY)

(tid, logicvalueid) -> ((tileRow, tileCol)-> ((offsetX, offsetY)

寄存器 ID

tileRow = ((logicalValueId / elementsPerRegister) % num8x128bTiles[0]) * **8**

分片行 ID

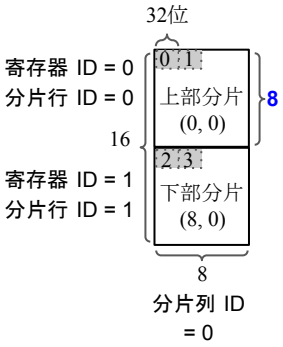
寄存器 ID

tileCol = ((logicalValueId / elementsPerRegister) / num8x128bTiles[0]) * **elementsPerLine**

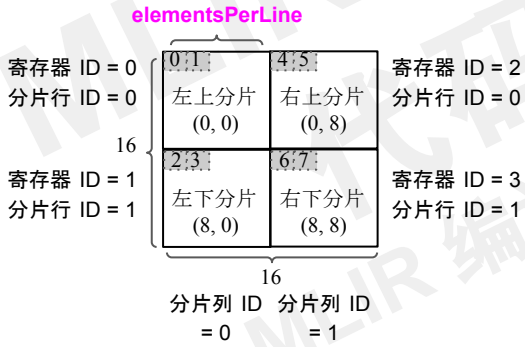
分片列 ID

(d0, d1) -> (tileRow, tileCol)
(d0, d1) -> (((d1 / 2) % 2) * **8**, ((d1 / 2) / 2) * **8**)

%C:16x8xi32

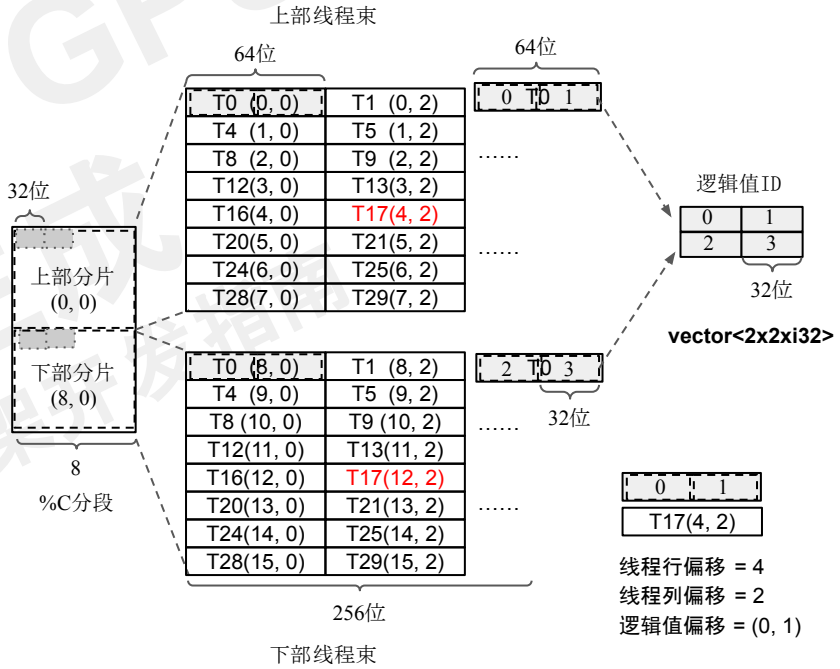


%C:16x16xi32



offsetX = **tileRow** + laneId / **kThreadsPerRow**
offsetY = **tileCol** + (laneId % **kThreadsPerRow**) * elementsPerRegister
+ (logicalValueIdDim % elementsPerRegister)}
线程行偏移 = laneId / **kThreadsPerRow**
线程列偏移 = (laneId % **kThreadsPerRow**) * elementsPerRegister
逻辑值偏移 = logicalValueId % elementsPerRegister

num8x128bTiles[0] = operandShape[0] / **kNumRowsPerTile**,
= 16 / **8** = 2
num8x128bTiles[1] = (operandShape[1] * elementType.getIntOrFloatBitWidth()) / lineSizeBits;
= 8 * 32 / 4 * 2 * 32 = 1

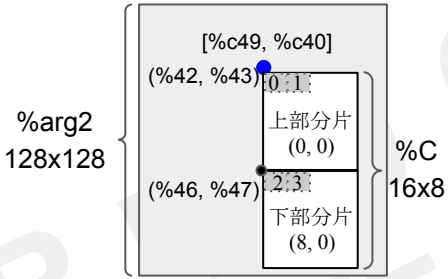


(d0, d1) -> (tileRow, tileCol)
(d0, d1) -> (((d1 / 2) % 2) * 8, ((d1 / 2) / 2) * 8)

(d0, d1) -> (offsetX, offsetY)
(d0, d1) -> (((d1 / 2) % 2) * 8 + d0 / 4, ((d1 / 2) / 2) * 8 + (d0 % 4) * 2 + d1 % 2)

线程加载数据的二维偏移量

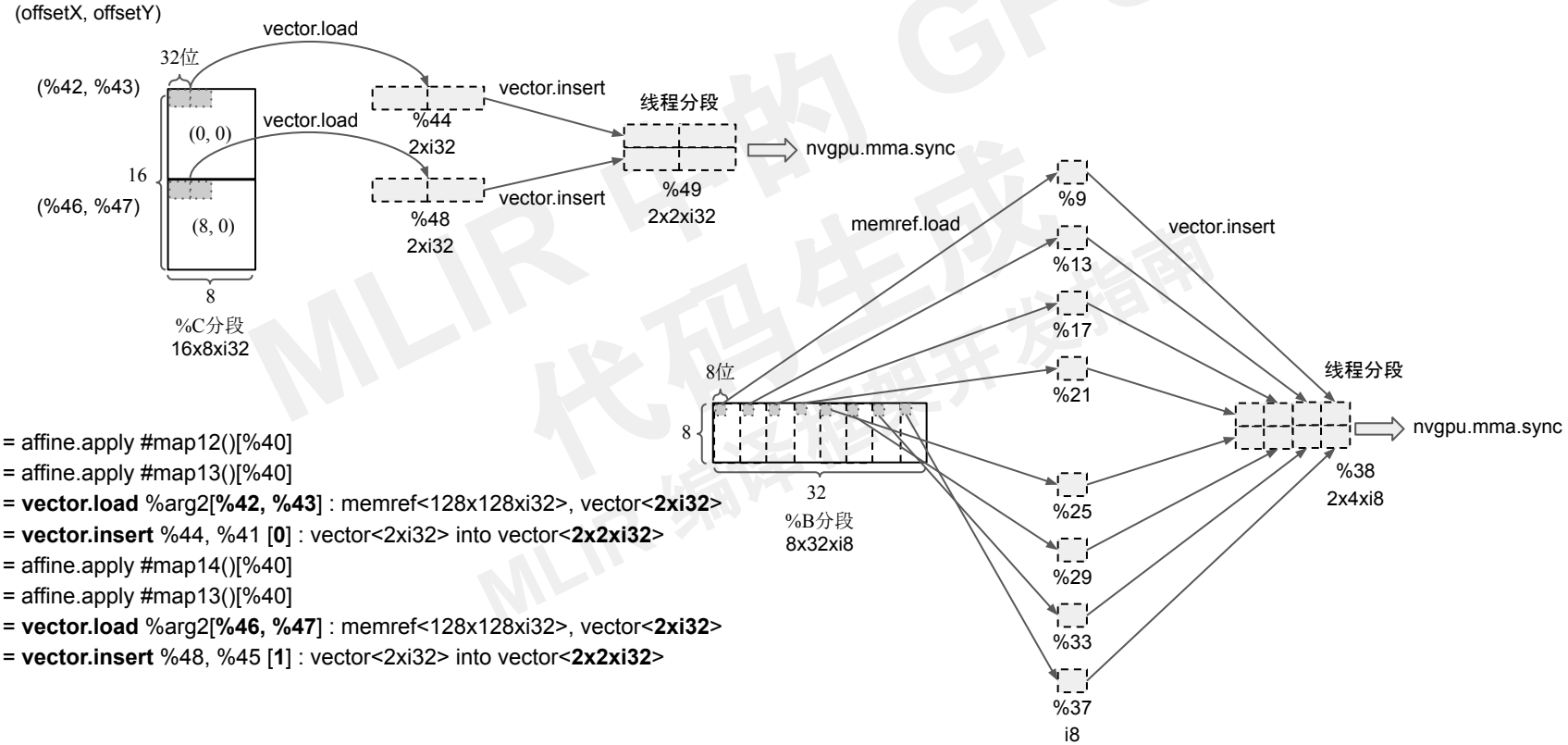
线程数据在原始操作数中的位置



%C = vector.transfer_read %arg2[%c49, %c40], %cst0 … :
memref<128x128xi32>, vector<16x8xi32>

%42 = affine.apply affine_map<()[s0] -> (s0 floordiv 4 + 49)>()[%40]
%43 = affine.apply affine_map<()[s0] -> (s0 * 2 - (s0 floordiv 4) * 8 + 40)>()[%40]
…
%46 = affine.apply affine_map<()[s0] -> (s0 floordiv 4 + 57)>()[%40]
%47 = affine.apply affine_map<()[s0] -> (s0 * 2 - (s0 floordiv 4) * 8 + 40)>()[%40]

vector.load + vector.insert 操作组合加载 %C 和 %B



%42 = affine.apply #map12()[%40]
%43 = affine.apply #map13()[%40]
%44 = **vector.load** %arg2[**%42, %43**] : memref<128x128xi32>, vector<**2xi32**>
%45 = **vector.insert** %44, %41 [**0**] : vector<2xi32> into vector<**2x2xi32**>
%46 = affine.apply #map14()[%40]
%47 = affine.apply #map13()[%40]
%48 = **vector.load** %arg2[**%46, %47**] : memref<128x128xi32>, vector<**2xi32**>
%49 = **vector.insert** %48, %45 [**1**] : vector<2xi32> into vector<**2x2xi32**>

4.3.4 vector.contract 操作的转换过程

vector.contract 操作的转换过程

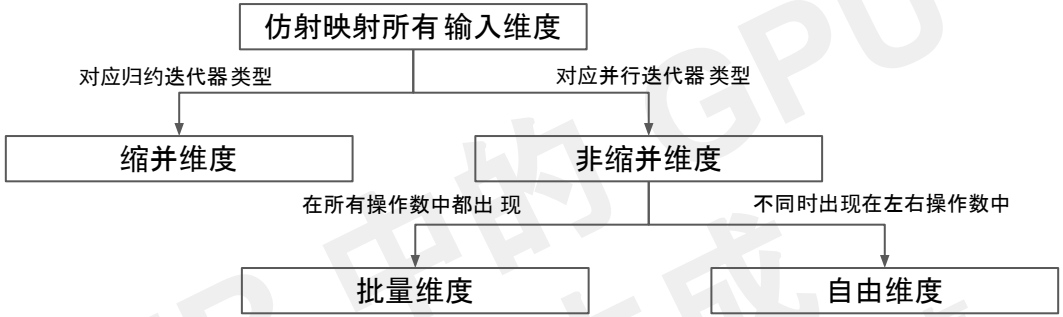
```
%acc  = arith.constant 0.0 : f32
%ones = arith.constant dense<1.0> : vector<8xf32>
%result = vector.contract {
 indexing_maps = [affine_map<(i) -> (i)>, affine_map<(i) -> (i)>, affine_map<(i) -> ()>],
 iterator_types = ["reduction"]
} %0, %ones, %acc : vector<8xf32>, vector<8xf32> into f32
```

```
for i in 0 to 8:
 %acc += %0[i] * %ones[i]
```

索引映射属性 indexing_maps 指定左、右输入操作数和累加器操作数的索引映射列表，列表中的每个仿射映射条目对应一个操作数，每个仿射映射条目用于表示在由归纳变量（或称为循环迭代器）构成的循环迭代空间的某个点上读写的操作数数据元素索引。

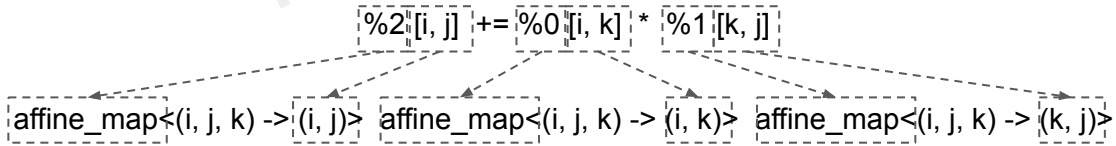
迭代器类型属性 iterator_types 指定了迭代器类型列表，列表中的每个元素代表一种迭代器类型，可以是归约（reduction）或并行（parallel）。

判断维度分类




```
#contraction_accesses = [
  affine_map<(i, j, k) -> (i, k)>, affine_map<(i, j, k) -> (k, j)>, affine_map<(i, j, k) -> (i, j)>
]
#contraction_trait = {
  indexing_maps = #contraction_accesses,
  iterator_types = ["parallel", "parallel", "reduction"]
}
%3 = vector.contract #contraction_trait %0, %1, %2
  : vector<4x3xf32>, vector<3x7xf32> into vector<4x7xf32>
```

```
for i in 0 to 4:
  for j in 0 to 7:
    for k in 0 to 3:
      %2[i, j] += %0[i, k] * %1[k, j]
```



判断维度分类

```
#contraction_accesses = [
  affine_map<(b0, f0, f1, c0, c1) -> (c0, b0, c1, f0)>,
  affine_map<(b0, f0, f1, c0, c1) -> (b0, c1, c0, f1)>,
  affine_map<(b0, f0, f1, c0, c1) -> (b0, f0, f1)>
]
#contraction_trait = {
  indexing_maps = #contraction_accesses,
  iterator_types = ["parallel", "parallel", "parallel", "reduction", "reduction"]
}

%4 = vector.contract #contraction_trait %0, %1, %2
    : vector<7x8x16x15xf32>, vector<8x16x7x5xf32> into vector<8x15x5xf32>
```

```
for b0 in 0 to 8:
  for f0 in 0 to 15:
    for f1 in 0 to 5:
      result = 0.0
      for c0 in 0 to 7:
        for c1 in 0 to 16:
          result += %0[c0][b0][c1][f0] * %1[b0][c1][c0][f1];
      %2[b0][f0][f1] = result;
```

vector.contract 操作到 nvgpu.mma.sync 操作的转换

convertContractOpToMmaSync( )

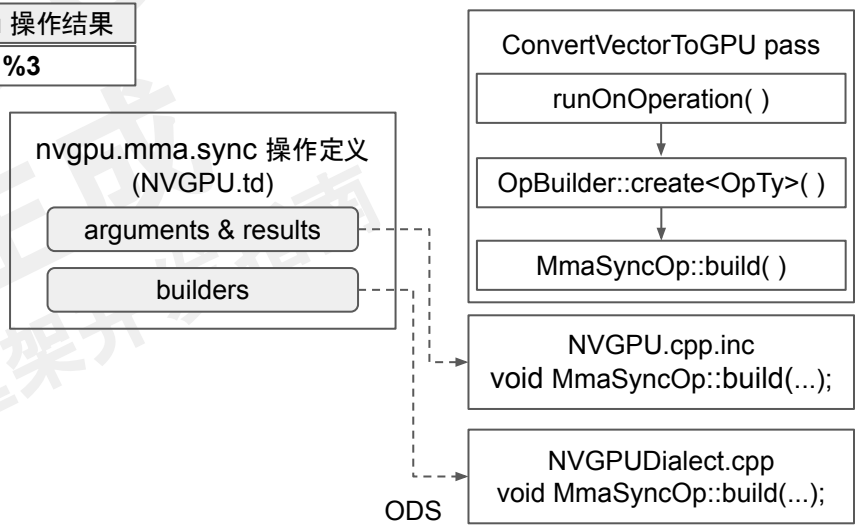
%A = vector.transfer_read %arg0 … ⇒ **%3** = nvgpu.ldmatrix %arg0 … ⇒ %51 = **nvgpu.mma.sync %3**, %38, %49 ... ⇐ %D = **vector.contract %A**, %B, %C…

valueMapping

vector操作结果	nvgpu 操作结果
%A	**%3**

NVGPU.td

```
def NVGPU_MmaSyncOp : NVGPU_MmaSyncOp<"mma.sync"> {
…
  let arguments = (ins AnyVector:$matrixA, AnyVector:$matrixB,
                       AnyVector:$matrixC, I64ArrayAttr:$mmaShape,
                       OptionalAttr<UnitAttr>:$tf32Enabled);
  let results = (outs AnyVector:$res);
  let builders = [
    OpBuilder<(ins "Value":$matrixA, "Value":$matrixB, "Value":$matrixC,
                   "ArrayAttr":$mmaShape)>,
    OpBuilder<(ins "Value":$matrixA, "Value":$matrixB, "Value":$matrixC,
                   "ArrayRef<int64_t>":$mmaShape, CArg<"bool",
"false">:$tf32Enabled)>
  ];
…
}
```



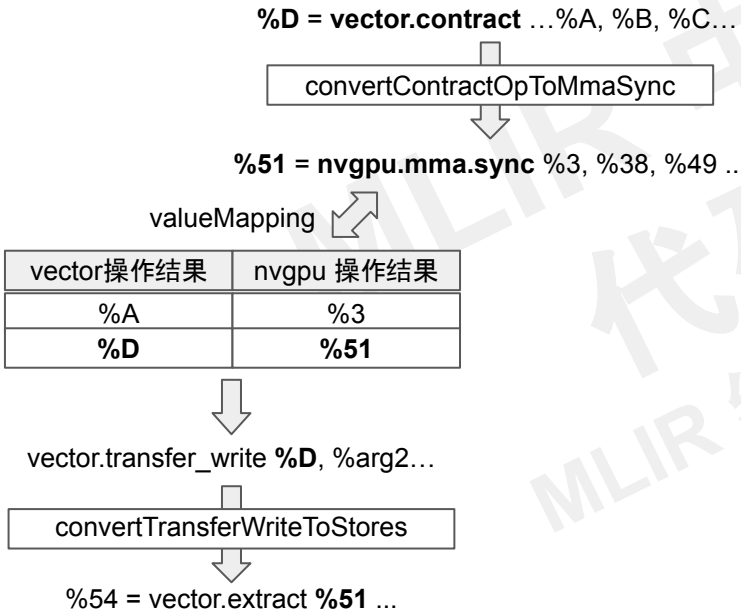
%51 = nvgpu.mma.sync(%3, %38, %49) {mmaShape = [16, 8, 32]} :
(vector<4x4xi8>, vector<2x4xi8>, vector<2x2xi32>) -> vector<2x2xi32>

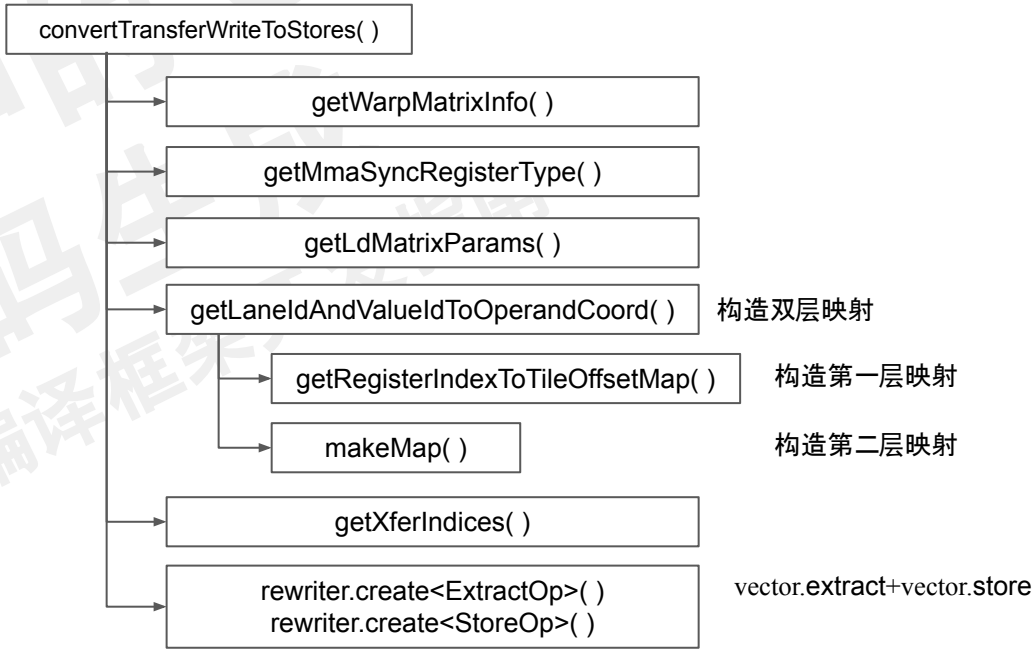
4.3.5 vector.transfer_write 操作的转换过程

vector.transfer_write 操作的转换过程

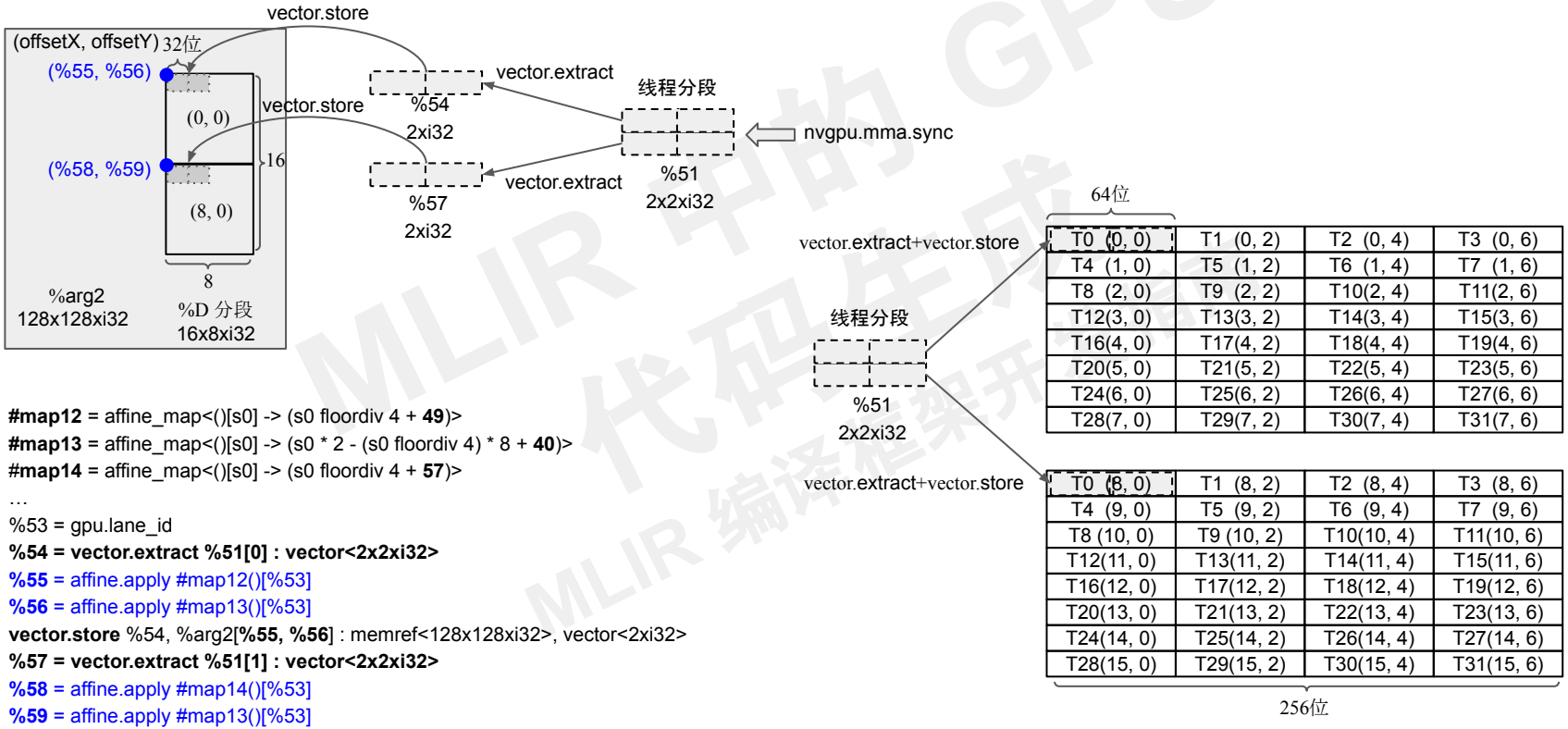
convertTransferWriteToStores( )

vector.transfer_write %D, %arg2[%c49, %c40] {in_bounds = [true, true]} :
vector<16x8xi32>, memref<128x128xi32>





vector.transfer_write 操作的转换过程



#map12 = affine_map<()[s0] -> (s0 floordiv 4 + **49**)>
#map13 = affine_map<()[s0] -> (s0 * 2 - (s0 floordiv 4) * 8 + **40**)>
#map14 = affine_map<()[s0] -> (s0 floordiv 4 + **57**)>
…
%53 = gpu.lane_id
%54 = vector.extract %51[0] : vector<2x2xi32>
%55 = affine.apply #map12()[%53]
%56 = affine.apply #map13()[%53]
vector.store %54, %arg2[**%55, %56**] : memref<128x128xi32>, vector<2xi32>
%57 = vector.extract %51[1] : vector<2x2xi32>
%58 = affine.apply #map14()[%53]
%59 = affine.apply #map13()[%53]
vector.store %57, %arg2[**%58, %59**] : memref<128x128xi32>, vector<2xi32>